

操作系统实验六：进程调度

姓名：蔡子轩
学号：2310984

目录

- [练习1：理解调度器框架的实现](#)
- [练习2：实现-round-robin-调度算法](#)
- [扩展练习-challenge-1实现-stride-scheduling-调度算法](#)
- [扩展练习-challenge-2在-ucore-上实现fifo与sjf](#)

练习1：理解调度器框架的实现

1) 调度类结构体 `sched_class` 的分析

问题：请详细解释 `sched_class` 结构体中每个函数指针的作用和调用时机；分析为什么需要将这些函数定义为函数指针，而不是直接实现函数。

解答：

`struct sched_class` 本质是“调度算法的统一接口表”（像 C++ 的 `vtable`），框架代码只认这 5 个接口，不直接写死 RR/Stride/FIFO/SJF 的细节，因此可以随时切换算法而不改调度主流程。

5 个函数指针含义与典型调用时机：

- `init(rq)`：初始化该算法需要的数据结构（如链表头/斜堆根、计数器等），在 `sched_init()` 时调用一次。
- `enqueue(rq, p)`：把就绪进程放入“就绪队列”（链表或堆），常见于进程创建/被唤醒/当前进程用完时间片后重新排队。
- `dequeue(rq, p)`：把某个进程从就绪队列移除，常见于它将要运行（被选中）或退出等。
- `pick_next(rq)`：从就绪队列里选“下一个运行的进程”（RR 取头、Stride 取最小 key）。
- `proc_tick(rq, current)`：每次时钟中断触发，用于维护时间片/设置 `need_resched`。

之所以做成函数指针：框架（机制）和算法（策略）分离——框架只调用 `sched_class->xxx()`，不关心底层是链表还是斜堆，也不关心选人规则，从而实现“易扩展、易切换”。

2) 运行队列结构体 `run_queue` 的分析

问题：比较 Lab5 和 Lab6 中 `run_queue` 的差异；解释为什么 Lab6 的 `run_queue` 需要支持两种数据结构（链表和斜堆）。

解答：

Lab5 的 RR 只要一个 FIFO 队列即可，所以 `run_queue` 里用 `run_list` 双向链表维护就绪进程，入队/出队都很简单。

Lab6 为了支持 Stride：调度时需要频繁找“最小 stride/pass 的进程”，如果仍用链表，每次找最小要扫一遍是 $O(N)$ ；因此新增 `lab6_run_pool`（斜堆/优先队列）来把“取最小”变成 $O(1)$ （堆顶），插入/删除均摊 $O(\log N)$ 。

为什么还要保留链表：RR/FIFO 这类算法只需要顺序轮转，链表的 $O(1)$ 操作更快更简单；如果强行用堆反而引入不必要的 $O(\log N)$ 开销。所以 Lab6 的 `run_queue` 做成“通用容器”，同时兼容链表算法与堆算法。

3) 调度器框架函数分析： `sched_init()` / `wakeup_proc()` / `schedule()`

问题：分析这三个函数在 Lab6 的实现变化，理解这些函数如何与具体调度算法解耦。

解答：

- `sched_init()`：Lab6 新增“绑定点”，核心是把全局 `sched_class` 指针指向某个具体实现（默认 RR / 你要切就换成 stride），然后调用 `sched_class->init(rq)` 完成该算法的队列初始化。
- `wakeup_proc()`：Lab5 只改状态为 `RUNNABLE`（因为它可能遍历全局进程链表选人）；Lab6 变为“状态 + 入队”，即除了设为 `RUNNABLE`，还会调用 `sched_class_enqueue(proc)`，真正入队逻辑由具体算法的 `enqueue` 决定（链表尾插还是斜堆插入）。
- `schedule()`：从“硬编码遍历/选择”改为“统一三步走”：`enqueue(current)` → `pick_next()` → `dequeue(next)`，最后 `proc_run(next)` 切换。框架只走流程，至于队列怎么维护、怎么选入，全交给 `sched_class` 的函数指针实现。

4) 调度类的初始化流程

问题：描述从内核启动到调度器初始化完成的完整流程；分析 `default_sched_class` 如何与调度器框架关联。

解答：

启动流程（抓主线即可）：`BootLoader` → `kern_entry` → `kern_init`；在 `kern_init` 中会调用 `sched_init()` 完成调度器初始化，然后才是 `proc_init()`、`clock_init()`，最后进入 `cpu_idle()` 等待首次调度。

`sched_init()` 内部的关键关联步骤是：

- 1) `sched_class = &default_sched_class`（把“接口指针”指向 RR 的实现）；2) 初始化全局 `rq` 并设置 `max_time_slice`；
- 3) 调用 `sched_class->init(rq)` 让算法初始化自己的数据结构。

因此 `default_sched_class` 与框架的关系就是：它实现了同一套 5 接口，并在初始化时被“挂到”全局 `sched_class` 上；之后框架统一通过 `sched_class->xxx()` 多态调用，不需要知道当前到底是 RR 还是 Stride。

5) 进程调度流程（含流程图）与 `need_resched` 作用

问题：绘制完整进程调度流程图（时钟中断 → `proc_tick` → `schedule()` → 调度类函数调用顺序），并解释 `need_resched` 标志位的作用。

解答：

流程图：

```
时钟中断到来
↓
trap() → interrupt_handler()
↓
```

```

sched_class_proc_tick() (调用当前算法的 proc_tick)
↓
time_slice--, 若减到 0 → need_resched = 1 (先“标记要调度”)
↓
中断处理结束, trap 返回前检查 need_resched
↓
if (need_resched) schedule()
↓
schedule(): enqueue(current) → pick_next() → dequeue(next) → proc_run(next)
↓
切换到 next 进程继续运行

```

这条链路里，“触发调度”的核心是：`proc_tick` 只负责在时间片用尽时把 `need_resched` 置 1；真正的进程切换发生在之后的 `schedule()`，并且调用顺序固定是 `enqueue → pick_next → dequeue → proc_run`。
`need_resched` 的作用可以一句话概括：**延迟调度**——在中断里只做“该换人了”的标记，等到安全位置再进入 `schedule()` 做完整队列操作和上下文切换，避免在中断处理中直接切换带来的复杂性。

6) 调度算法的切换机制（如何接入新算法，如 Stride）

问题：如果要添加一个新的调度算法（如 stride），需要修改哪些代码？为什么当前设计使切换变得容易？

解答：

添加/切换一个新算法，核心就是“实现接口 + 换指针”：

1) 新增/实现算法文件（例如 `default_sched_stride.c`）：实现 5 个接口函数，并定义 `stride_sched_class` 实例把函数指针填进去；

2) 在 `default_sched.h` 里 `extern` 声明该调度类；

3) 在 `sched_init()`（`sched.c`）里把 `sched_class = &default_sched_class` 改成 `sched_class = &stride_sched_class`（切换算法只需改这一行）。

之所以容易：框架层（`schedule/wakeup_proc/proc_tick` 调用链）只依赖统一接口，策略层（RR/Stride/FIFO/SJF）可替换；“机制负责何时调度，策略负责选谁调度”，互不影响。

练习2：实现 Round Robin 调度算法

(1) Lab5 vs Lab6: `wakeup_proc()` 的改动（为什么要显式入队）

问题：比较一个在 Lab5 和 Lab6 都有、但实现不同的函数（如 `kern/schedule/sched.c` 里的 `wakeup_proc()`），说明为什么要做这个改动；如果不做会有什么问题？（要求：列出两者代码对比）

① Lab5 的 `wakeup_proc()`（只改状态）

```

void wakeup_proc(struct proc_struct *proc) {
    if (proc->state != PROC_RUNNABLE) {
        proc->state = PROC_RUNNABLE; // 仅修改状态
        proc->wait_state = 0;
    }
}

```

② Lab6 的 `wakeup_proc()` (改状态 + 显式入队)

```
void wakeup_proc(struct proc_struct *proc) {
    if (proc->state != PROC_RUNNABLE) {
        proc->state = PROC_RUNNABLE;
        proc->wait_state = 0;
        if (proc != current) {
            sched_class_enqueue(proc); // ★ 新增：显式入队
        }
    }
}
```

为什么要改？

• Lab5：状态驱动

Lab5 的调度过程会遍历全局进程链表 (`proc_list`)，只要发现 `state == PROC_RUNNABLE` 的进程就能被选中运行。因此 `wakeup_proc()` 只需要把进程状态改成 `RUNNABLE`，进程就会在下一次遍历时“自然被找到”。

• Lab6：队列驱动

Lab6 引入独立的就绪队列 `run_queue`（只存放就绪进程），并且 `schedule()` 不再遍历 `proc_list`，而是调用 `sched_class_pick_next()` 只从 `run_queue` 里选下一个进程。

因此唤醒时必须把进程显式入队，否则它虽然变成 `RUNNABLE`，但不在 `run_queue`，调度器“看不见它”。

不改会出什么问题？

如果 Lab6 的 `wakeup_proc()` 仍像 Lab5 那样只改状态、不 `enqueue`：

- 被唤醒/新创建的进程不会进入 `run_queue`；
- `schedule()` 调用 `pick_next()` 时队列可能为空；
- 结果该进程永远不会被调度执行，严重时表现为系统卡住或功能异常（例如 `fork()` 出来的子进程一直不运行）。

(2) RR 调度器实现：函数思路、链表操作选择与边界情况

问题：描述你实现每个函数的具体思路和方法，解释为什么选择特定的链表操作方法。对每个实现函数的关键代码进行解释说明，并解释如何处理边界情况。

解答：

RR 的核心就是“队尾入队、队头出队”的 FIFO 轮转：`run_queue` 里维护一个双向循环链表 `run_list`，就绪进程用 `proc->run_link` 挂在链表上；`pick_next()` 永远取 `run_list` 的第一个元素作为“队头”，而 `enqueue()` 把新进程插到 `run_list` 之前作为“队尾”，这样当一个进程用完时间片被重新入队时，会自然排到最后，实现公平轮转。实现上我把关键点集中在两个地方：一是链表结构不被破坏（重复入队、错误出队要尽早报错），二是时间片字段的健壮性（新进程/用完时间片要补满，被抢占但没用完要保留，且避免 unsigned 下溢）。
:contentReference[oaicite:0]{index=0}

1) RR_init：初始化空队列

RR_init() 很直接：把 run_list 初始化成“自己指向自己”的空循环链表，同时把 proc_num 清零即可；max_time_slice 由框架在 sched_init() 里统一设置，所以这里不重复初始化。:contentReference[oaicite:1]{index=1}

```
static void RR_init(struct run_queue *rq) {
    list_init(&(rq->run_list));
    rq->proc_num = 0;
}
```

2) RR_enqueue: 入队（队尾插入）+ 时间片修正 + 防重复入队

RR_enqueue() 是 RR 最关键的函数：

- (1) 先断言 run_link 为空，防止同一个进程被重复入队破坏链表结构（重复入队会直接触发 assert，定位 bug 很快）；
- (2) 用 list_add_before(&(rq->run_list), &(proc->run_link)) 把节点插到表头之前，等价于插到队尾，配合“队头取出”实现 FIFO；
- (3) 时间片处理：如果 time_slice==0（新建进程/用完时间片）或 time_slice > max_time_slice（异常值）就补满为 max_time_slice；否则（比如被更高优先级抢占，还剩 2 tick）就保留剩余时间片，体现抢占式下的“剩余片不丢”。最后设置 proc->rq=rq 并递增计数。

```
static void RR_enqueue(struct run_queue *rq, struct proc_struct *proc) {
    assert(list_empty(&(proc->run_link))); // 防重复入队
    list_add_before(&(rq->run_list), &(proc->run_link)); // 插队尾: FIFO

    if (proc->time_slice == 0 || proc->time_slice > rq->max_time_slice) {
        proc->time_slice = rq->max_time_slice; // 新进程/用完片/异常值: 补满
                                                // 否则保留剩余片（被抢占场景）
    }

    proc->rq = rq;
    rq->proc_num++;
}
```

3) RR_dequeue: 出队（删除并“重新初始化节点”）

RR_dequeue() 的目标是把指定进程从就绪队列移除，同时保证它之后还能安全再次入队：

- (1) 断言两件事：run_link 不为空（说明确实在某个队列里）且 proc->rq == rq（防止从错误的 run_queue 出队）；
- (2) 用 list_del_init(&proc->run_link) 而不是 list_del()：list_del_init 在断链后会把该节点的 next/prev 重新指向自己，这样不会留下悬空指针，并且后续还能用 list_empty(&proc->run_link) 来判断“当前是否在队列中”，也保证该节点可被再次安全入队；
- (3) proc_num--。

```
static void RR_dequeue(struct run_queue *rq, struct proc_struct *proc) {
    assert(!list_empty(&(proc->run_link)) && proc->rq == rq);
    list_del_init(&(proc->run_link)); // 删除 + 初始化, 便于再次入队/检测在队列中
    rq->proc_num--;
}
```

4) RR_pick_next: 选队头 (空队列返回 NULL)

RR_pick_next() 只做“取队头”这件事: 用 list_next(&run_list) 得到表头后的第一个节点; 如果它又等于 &run_list, 说明队列为空返回 NULL; 否则用 le2proc(le, run_link) 把链表节点地址换回进程结构体地址。这样可以自然覆盖三种边界: 空队列、单进程、多进程轮转。

```
static struct proc_struct *RR_pick_next(struct run_queue *rq) {
    list_entry_t *le = list_next(&(rq->run_list));
    if (le == &(rq->run_list)) return NULL; // 空队列
    return le2proc(le, run_link); // 队头进程
}
```

5) RR_proc_tick: 时钟到来时扣时间片, 并“延迟触发调度”

RR_proc_tick() 每个 tick 都会执行: 如果 time_slice > 0 才做 --, 这是为了防止 time_slice 是 unsigned 时在 0 上继续 -- 发生下溢变成超大正数; 当 time_slice == 0 时只设置 need_resched=1, 表示“请求调度”, 但不在中断上下文里直接切换进程, 真正的 schedule() 会在安全位置 (trap 返回前) 检查到该标志后再执行。

```
static void RR_proc_tick(struct run_queue *rq, struct proc_struct *proc) {
    if (proc->time_slice > 0) {
        proc->time_slice--;
    }
    if (proc->time_slice == 0) {
        proc->need_resched = 1; // 延迟调度: 只做标记
    }
}
```

6) 我在 RR 中重点处理的边界情况

- 重复入队: assert(list_empty(&proc->run_link)) 直接拦住“同一节点二次挂链”这种会把链表搞坏的致命 bug。
- 错误出队 / 重复出队: assert(proc->rq == rq && !list_empty), 保证从“正确的队列”移除“确实在队列里的节点”。
- 时间片异常: time_slice==0 补满、time_slice>max 纠正, 抢占回队则保留剩余片。
- unsigned 下溢: tick 里 if (time_slice > 0) time_slice--;, 确保 0 不会减成超大值。
- 空队列: pick_next() 检测 list_next==head 返回 NULL, 由框架选择 idleproc 或做相应处理。

(3) 实验结果展示：make grade 与 QEMU 运行现象分析

问题：展示 make grade 的输出结果，并描述在 QEMU 中观察到的调度现象（包括 make grade 结果、make qemu 结果，并进行一定分析说明实验成功）。

① make grade 结果（自动评测通过）

从评测输出可以看到 priority 测例的结果与输出检查均为 OK，总分 50/50，说明本次 RR 调度相关实现满足评测要求。

```
● wsy@sparewheel:~/oslab/labcode/labcode/lab6$ make grade
priority: (3.1s)
  -check result: OK
  -check output: OK
Total Score: 50/50
```

② make qemu 运行现象（调度行为观察）

在 QEMU 中运行 priority 用户程序时，可以观察到：程序创建多个子进程并设置不同的 priority（如 1~6），随后进行固定 tick 数（如 100 ticks）的统计，最后输出每个进程的累计执行情况（acc、time）以及最终的调度结果汇总。


```
kernel_execve: pid = 2, name = "priority".
set priority to 6
main: fork ok,now need to wait pids.
set priority to 1
set priority to 2
set priority to 3
set priority to 4
set priority to 5
100 ticks
End of Test.
100 ticks
End of Test.
child pid 3, acc 992000, time 2010
child pid 4, acc 976000, time 2010
child pid 5, acc 964000, time 2010
child pid 6, acc 956000, time 2010
child pid 7, acc 976000, time 2010
main: pid 0, acc 992000, time 2010
main: pid 4, acc 976000, time 2010
main: pid 5, acc 964000, time 2010
main: pid 6, acc 956000, time 2010
main: pid 7, acc 976000, time 2010
main: wait pids over
sched result: 1 1 1 1 1
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:538:
    initproc exit.
```

③ 现象分析（说明 RR 调度实现正确）

- RR 的核心特征是：忽略 priority 等“权重/优先级”信息，对所有可运行进程进行时间片轮转，尽量做到公平。
- 从 QEMU 输出可以看到：虽然程序给不同子进程设置了不同的 priority，但最终统计中各个子进程的 `time` 相同（均为同一数量级/相同 tick 统计区间），`acc` 也非常接近；并且最后打印的 `sched result: 1 1 1 1 1` 表明各进程获得的调度份额基本一致，符合 RR 的“公平轮转”预期。
- 测试结束后出现 `kernel panic ... initproc exit` 属于 uCore 实验常见的收尾现象：用户态测试进程退出、`initproc` 结束后内核进入预期的终止路径并触发 panic 信息，前面的评测与运行输出已表明调度功能

正确完成。

(4) Round Robin (RR) 调度算法分析：优缺点、时间片选择与 `need_resched`

问题：分析 Round Robin 调度算法的优缺点，讨论如何调整时间片大小来优化系统性能，并解释为什么需要在 `RR_proc_tick` 中设置 `need_resched` 标志。

解答：

RR（时间片轮转）的优点是**实现简单、对所有进程公平**，并且响应时间相对可预测；本实验中多个进程的运行统计较为接近，也体现了 RR 的公平性。其缺点是**不区分优先级/紧急程度**，难以满足实时性或优先级需求；同时在 I/O 密集型与交互型场景下，如果时间片选择不当，仍可能出现响应不够好或切换开销过大等问题。

时间片大小存在典型权衡：**时间片过大会**让交互响应变差，整体行为接近 FIFO；**时间片过小**会导致频繁上下文切换，切换开销占比上升、CPU 有效利用率下降（例如时间片仅 1 tick，而切换开销 0.5 tick，则有效利用率约为 $1/(1+0.5)=67\%$ ）。因此时间片常选在“既能保证交互响应、又不过度切换”的区间，经验上可取为**上下文切换时间的 10 倍左右**；交互系统常见 10-100ms，偏批处理可放宽到 100-1000ms（实验中取 5 tick 也属于折中选择）。

最后，`RR_proc_tick` 中设置 `need_resched` 的原因是把“**检测是否该调度**”与“**真正执行调度**”解耦：`proc_tick` 在时钟中断上下文执行，此时不适合直接做完整的 `schedule()`（队列操作/进程切换会更复杂且易出错），因此只置位 `need_resched=1`，让系统在中断返回前的安全点进入 `schedule()` 完成切换；同时这也带来职责清晰与复用性——除了时钟到期外，`do_yield` 等路径也可以通过置位该标志触发调度。

(5) 拓展思考：优先级 RR 与多核调度支持

问题：如果要实现优先级 RR 调度，你的代码需要如何修改？当前的实现是否支持多核调度？如果不支持，需要如何改进？

解答：

优先级 RR 可以理解为“**优先级队列之间按优先级选人，每个队列内部仍用 RR 轮转**”。实现上要把当前的“单一 `run_list`”改成“**多级队列**”：在 `run_queue` 中用数组保存多个链表（每个优先级一个就绪队列）以及对应计数器，同时维护一个 `highest_priority`（或类似字段）用于加速 `pick_next` 查找。入队时根据 `proc->priority` 把进程挂到对应优先级队列尾部，并在必要时更新 `highest_priority`；出队时从对应优先级队列删除，若该队列变空则需要重新寻找当前最高的非空优先级。`pick_next` 则从最高优先级的队列取队首进程运行，而同一优先级内仍遵循“队尾入队、队头取出”的 RR 行为。`proc_tick` 一般不必大改，因为时间片机制仍是“用完置位 `need_resched`”这一套；但需要注意这种策略可能导致低优先级进程**饥饿**，工程上通常会加入“**优先级提升/aging**”：例如等待超过阈值（如 1000 ticks）的进程临时提升优先级，运行后再恢复，从而兼顾响应性与公平性。

多核调度 方面，当前实现一般不支持（或扩展性很差），原因是使用“**全局唯一 `run_queue`**”：多 CPU 并发调度时需要争用同一把锁，锁竞争会非常严重，核数一多就会成为瓶颈。改进的经典方案是“**per-CPU `run_queue`**”：为每个 CPU 维护独立的运行队列和锁（例如 `per_cpu_rq[]`），调度时只锁本 CPU 的队列，从而显著降低锁竞争并提升可扩展性。但这会带来**负载不均衡**，因此还要增加负载均衡器：周期性检查各 CPU 的就绪进程数量，差距超过阈值就迁移部分进程；迁移时需要双锁协议并按 CPU ID 顺序加锁避免死锁。同时还可考虑**CPU 亲和性**：尽量让进程在同一 CPU 上运行以提高缓存命中率；当某 CPU 空闲时，可通过 IPI 唤醒空闲核参与调度新任务。整体思路与 Linux 的多核调度架构类似。

扩展练习 Challenge 1: 实现 Stride Scheduling 调度算法

(1) Stride Scheduling 代码实现分析（按函数说明）

下面我将结合我在 `default_sched_stride.c` 中的具体实现，列出关键代码（去掉注释），并说明各函数的设计思路、关键点与边界情况处理。

① 全局常量与比较函数： `BIG_STRIDE` + `proc_stride_comp_f`

```
#include <defs.h>
#include <list.h>
#include <proc.h>
#include <assert.h>
#include <default_sched.h>
#include <stdio.h>

#define USE_SKEW_HEAP 1
#define BIG_STRIDE 0x7FFFFFFF

static int proc_stride_comp_f(void *a, void *b) {
    struct proc_struct *p = le2proc(a, lab6_run_pool);
    struct proc_struct *q = le2proc(b, lab6_run_pool);
    int32_t c = (int32_t)(p->lab6_stride - q->lab6_stride);
    if (c > 0) return 1;
    else if (c < 0) return -1;
    else return 0;
}
```

分析：

- `BIG_STRIDE` 取 `0x7FFFFFFF` ($2^{31}-1$)，用来计算步长 `pass += BIG_STRIDE / priority`，既能保证步长精度足够大，又为“溢出比较”预留了半圈空间（后面用 `int32_t` 差值比较）。
- `proc_stride_comp_f` 是斜堆的比较器：把斜堆节点指针还原为进程指针后，用 `int32_t(p->lab6_stride - q->lab6_stride)` 的符号来判断大小。这样即便 `lab6_stride` 发生无符号回绕（溢出），只要任意两者差值不跨越半圈 ($<2^{31}$)，比较仍然稳定正确。

② 初始化： `stride_init`

```
static void stride_init(struct run_queue *rq) {
    list_init(&(rq->run_list));
    rq->lab6_run_pool = NULL;
    rq->proc_num = 0;
}
```

分析：

Stride 的就绪队列主要靠 斜堆（优先队列）维护“最小 pass 的进程”，因此初始化时把堆根 `lab6_run_pool` 置空、进程数清零即可；`run_list` 这里仍做初始化是为了保持 `run_queue` 结构通用（同一框架下可支持 RR/Stride 等不同实现），但本实现的核心选人逻辑不依赖链表。

③ 入队： `stride_enqueue`

```
static void stride_enqueue(struct run_queue *rq, struct proc_struct *proc) {
    rq->lab6_run_pool = skew_heap_insert(rq->lab6_run_pool, &(amp;proc->lab6_run_pool),
    proc_stride_comp_f);

    if (proc->time_slice == 0 || proc->time_slice > rq->max_time_slice) {
        proc->time_slice = rq->max_time_slice;
    }

    proc->rq = rq;
    rq->proc_num++;
}
```

分析：

- 入队的关键是把 `proc->lab6_run_pool` 节点插入斜堆，使堆顶永远是 `lab6_stride` 最小的进程（也就是下一次最应该被调度的进程）。插入使用 `skew_heap_insert`，复杂度均摊 $O(\log N)$ 。
- 时间片策略与 RR 一致：如果进程 `time_slice==0`（新建/刚用完）或出现异常大于 `max_time_slice`，就统一重置为 `max_time_slice`，确保每次被调度都有可用时间片。
- 同时维护 `proc->rq` 和 `rq->proc_num`，保证框架统计与队列状态一致。

④ 出队： `stride_dequeue`

```
static void stride_dequeue(struct run_queue *rq, struct proc_struct *proc) {
    rq->lab6_run_pool = skew_heap_remove(rq->lab6_run_pool, &(amp;proc->lab6_run_pool),
    proc_stride_comp_f);
    rq->proc_num--;
}
```

分析：

- 进程被真正选中运行前，框架会调用 `dequeue` 把它从就绪结构中移除（与 RR 一样：`pick_next` 负责“选”，`dequeue` 负责“摘”）。
- 这里用 `skew_heap_remove` 删除指定节点，并更新堆根指针；删除后 `proc_num--`。
- 这种“先 pick，再 dequeue”的分工，使得调度框架与具体策略保持一致的调用顺序（便于算法替换/复用）。

⑤ 选下一个进程并更新 pass： `stride_pick_next`

```
static struct proc_struct *stride_pick_next(struct run_queue *rq) {
    if (rq->lab6_run_pool == NULL) return NULL;

    struct proc_struct *p = le2proc(rq->lab6_run_pool, lab6_run_pool);

    if (p->lab6_priority == 0) {
        p->lab6_stride += BIG_STRIDE;
    } else {
        p->lab6_stride += BIG_STRIDE / p->lab6_priority;
    }

    return p;
}
```

分析：

- 斜堆的堆顶就是“当前 pass 最小”的进程，因此直接 `le2proc(rq->lab6_run_pool, lab6_run_pool)` 得到要运行的进程指针，选人是 $O(1)$ 。
- 选中后立刻更新它的 `lab6_stride`（这里的 `lab6_stride` 实际上扮演 **pass** 的角色）：
 - 每运行一次，就加上该进程的步长 `BIG_STRIDE / priority`。
 - `priority` 越大，步长越小，pass 增长越慢，因此它更容易保持“较小 pass”，从而在长期获得更多 CPU 份额。
- `priority==0` 的分支属于防御性处理（正常实验设定一般 `priority>=1`），避免除 0 直接崩溃。

⑥ 时钟 tick: `stride_proc_tick`（时间片耗尽触发调度）

```
static void stride_proc_tick(struct run_queue *rq, struct proc_struct *proc) {
    if (proc->time_slice > 0) {
        proc->time_slice--;
    }
    if (proc->time_slice == 0) {
        proc->need_resched = 1;
    }
}
```

分析：

- 逻辑与 RR 基本一致：每个 tick 扣减时间片，减到 0 时只设置 `need_resched=1`，表示“需要切换”，真正的 `schedule()` 会在安全点统一完成选人、出队、切换等操作。
- `if (time_slice > 0)` 是为了避免 `time_slice` 为无符号时在 0 上继续 `--` 下溢变成超大值，导致永远不会触发调度。

⑦ 调度类注册: `stride_sched_class`

```
struct sched_class stride_sched_class = {  
    .name = "stride_scheduler",  
    .init = stride_init,  
    .enqueue = stride_enqueue,  
    .dequeue = stride_dequeue,  
    .pick_next = stride_pick_next,  
    .proc_tick = stride_proc_tick,  
};
```

分析：

这一结构体把 Stride 的 5 个核心接口挂到调度框架上。完成后只需要在 `sched_init()` 中把 `sched_class` 指针切换到 `&stride_sched_class`，框架层（`wakeup_proc/schedule/proc_tick` 调用链）无需再改，就能整体切换到 Stride 策略。

（2）结果验证：QEMU 输出与调度现象分析（Stride）

问题：展示 `make qemu` 的输出结果，并结合输出说明 Stride Scheduling 的调度现象，证明实现有效。

① `make qemu` 关键输出

```
kernel_execve: pid = 2, name = "priority".
set priority to 6
main: fork ok,now need to wait pids.
set priority to 5
set priority to 4
set priority to 3
set priority to 2
set priority to 1
100 ticks
End of Test.
100 ticks
End of Test.
child pid 7, acc 1396000, time 2010
child pid 6, acc 1136000, time 2010
child pid 5, acc 904000, time 2010
child pid 4, acc 664000, time 2010
child pid 3, acc 428000, time 2010
main: pid 3, acc 428000, time 2010
main: pid 4, acc 664000, time 2010
main: pid 5, acc 904000, time 2010
main: pid 6, acc 1136000, time 2010
main: pid 0, acc 1396000, time 2020
main: wait pids over
sched result: 1 2 2 3 3
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:538:
  initproc exit.
```

② 现象分析（体现“按优先级比例分配 CPU”）

从截图可以观察到 `priority` 用户程序创建多个子进程后，为它们设置了不同的 `priority`（从 6 到 1），随后在固定 tick 统计窗口内（如 `100 ticks`）累计每个进程的执行量，并在结束时打印各进程的 `acc`（累计执行次数/工作量）与 `time`（统计时间）。在 Stride 调度下，虽然所有进程的 `time` 基本一致（都在同一统计区间内运行），但各进程的 `acc` 明显呈现“**priority 越大，acc 越高**”的趋势：例如截图中最高优先级进程的 `acc` 明显大于低优先级进程（从 42.8 万逐步上升到 139.6 万），这符合 Stride 的设计目标——通过 `pass += BIG_STRIDE / priority` 让高 `priority` 的进程步长更小、`pass` 增长更慢，从而在长期内被更频繁地选中运行，获得更高 CPU 份额。

此外，末尾的 `sched_result: 1 2 2 3 3` 也说明调度器对不同优先级的进程做出了区分：相比 RR 中“各进程几乎相同”的结果（常见为全 1），Stride 下出现不同的统计等级（2、3 等），进一步表明“调度份额不再均匀，而是按 `priority` 倾斜”。整体输出与 Stride 的预期一致，说明实现正确并且成功影响了实际调度行为。

最后出现的 `kernel panic ... initproc exit` 是 uCore 实验环境的常见收尾现象：用户态测试退出后进入预期终止路径触发 `panic` 信息，不影响对调度现象与实现正确性的判断。

（3）拓展设计：多级反馈队列（MLFQ）调度算法的概要设计与详细设计

问题：简要说明如何设计实现“多级反馈队列调度算法（MLFQ）”，给出概要设计，鼓励给出详细设计。

回答：

1) 概要设计（核心思想）

MLFQ 的核心是维护 **K 个按优先级从高到低排列的就绪队列**，调度时始终从**最高优先级的非空队列**取队首进程运行。每个进程在 PCB 中记录自己当前所在的队列层级（`level`）以及剩余时间片（`time_slice`），并且**不同层级配置不同的时间片长度**：高优先级层时间片短、低优先级层时间片长，从而实现“交互优先、CPU 密集型逐步下沉”的反馈机制。

2) 详细设计（数据结构与规则）

- **数据结构**：在 `run_queue` 中维护一个长度为 K 的队列数组 `rq->queues[K]`（每层用 FIFO 链表即可），并记录每层的时间片配置 `quantum[K]`；在 `proc_struct` / PCB 中新增 `mlfq_level`（当前层级）。
- **选进程规则（pick_next）**：从 `level=0`（最高）开始向下扫描，找到第一个非空队列，取其队首作为 `next` 运行。
- **时间片与反馈（proc_tick）**：每次时钟 tick 递减当前进程 `time_slice`。若时间片耗尽且进程仍处于 `RUNNABLE`，说明更偏 CPU-bound，则将其 **level 下调一级**（若已在最低层则保持不变）并重新入队；若进程在时间片未用完时**主动阻塞/让出**（如 I/O 等待），则通常**保持当前层级**，体现对交互型任务的倾斜。
- **防饥饿策略（priority boost）**：为避免低优先级长期得不到 CPU，可设计**周期性提升**：每隔固定 tick，将所有就绪进程统一提升回最高优先级队列并重置时间片，保证系统整体公平性。
- **I/O 进程唤醒策略（wakeup/enqueue）**：对于 I/O 密集型进程，唤醒时可以直接插入高优先级队列（或至少不低于当前层），使其快速响应。

3) 落到 uCore 调度框架的实现位置（对应接口怎么配合）

在现有 `sched_class` 框架下实现 MLFQ，只需要用接口把“机制”和“策略”接起来即可：

- `init(rq)`：初始化 K 条 FIFO 队列、清空计数器、设置各层时间片参数；
- `enqueue/dequeue`：按 `proc->mlfq_level` 将进程插入/移除对应层队列（一般入队插队尾，保持 FIFO）；
- `pick_next`：从高到低扫描队列选择队首；
- `proc_tick`：递减 `time_slice`，时间片用尽触发降级、重置时间片、并设置 `need_resched`；额外在此处

维护 boost 计数器，到点执行全局提升。

(4) 原理说明：为什么 Stride 调度下“时间片份额 $\propto$ 优先级”

问题：简要证明/说明（不必特别严谨，但应当能够“说服你自己”），为什么 Stride 算法中，经过足够多的时间片之后，每个进程分配到的时间片数目和优先级成正比。

回答：

在 Stride 调度中，每个进程维护一个累计量 `pass`（我代码里用 `lab6_stride` 表示），调度器每次都选择 `pass` 最小的进程运行一次。进程每运行一次，就把自己的 `pass` 增加一个固定步长 `stride`，且步长由优先级决定：

$$stride = \frac{BIG}{priority}$$

直观理解是：**priority 越大 $\rightarrow$ stride 越小 $\rightarrow$ 每跑一次 pass 增得越慢 $\rightarrow$ 更容易一直保持在“最小 pass”附近 $\rightarrow$ 更容易被频繁选中运行**，因此获得更多 CPU 份额。

更形式化一点，用两个进程 A、B 来看：它们的步长分别是

$$s_A = \frac{BIG}{p_A}, \quad s_B = \frac{BIG}{p_B}$$

如果在一段足够长的时间里，A 被调度运行了 (n_A) 次、B 被调度运行了 (n_B) 次，则它们的累计 pass 大致为

$$pass_A \approx n_A \cdot s_A, \quad pass_B \approx n_B \cdot s_B$$

由于调度器总是选择最小 pass，领先太多的进程会暂时“跑不到”（不容易被选中），落后的进程会不断获得运行机会来“追赶”，因此长期统计下各进程的 pass 会被拉在同一量级上，不会无限拉开。于是可以用“量级相当”的近似关系表示为：

$$n_A \cdot s_A \approx n_B \cdot s_B$$

两边同时除以 $n_B \cdot s_A$ ，得到：

$$\frac{n_A}{n_B} \approx \frac{s_B}{s_A} = \frac{BIG/p_B}{BIG/p_A} = \frac{p_A}{p_B}$$

这说明：运行次数（拿到的时间片数）之比近似等于优先级之比。推广到多个进程同理：所有进程的 `pass` 都被“拉齐”在相近水平上，而步长越小（优先级越高）的进程为了追到同一 pass 水平需要更多次运行，因此在长期内自然获得更多时间片，实现按权重（优先级）近似成比例的 CPU 分配。

扩展练习 Challenge 2：在 uCore 上实现 FIFO 与 SJF

(1) FIFO 调度算法实现分析（按函数说明）

接下来我讲给出 FIFO 调度器的关键实现代码，并按函数说明设计思路、关键点与边界情况处理。

① FIFO_init：初始化运行队列

```
static void FIFO_init(struct run_queue *rq) {
    list_init(&(rq->run_list));
    rq->proc_num = 0;
}
```

分析：

FIFO 采用最简单的就绪队列组织方式：一个 FIFO 队列即可，因此 `init` 只需要把 `run_list` 初始化为空循环链表（头结点前后指针指向自己），并把进程计数 `proc_num` 清零。该初始化保证后续 `pick_next` 能通过“`list_next(head)==head` 判空”的方式处理空队列边界情况。

② FIFO_enqueue：入队（队尾插入）

```
static void FIFO_enqueue(struct run_queue *rq, struct proc_struct *proc) {
    assert(list_empty(&(proc->run_link)));
    list_add_before(&(rq->run_list), &(proc->run_link));
    proc->rq = rq;
    rq->proc_num++;
}
```

分析：

- FIFO 的队列语义是“先来先服务”，因此入队时必须插入到队尾。本实现使用 `list_add_before(&rq->run_list, &proc->run_link)`，把新节点插在链表头之前，等价于插到队尾（因为 `run_list` 是哨兵头结点）。
- `assert(list_empty(&proc->run_link))` 用于防止同一进程重复入队破坏链表结构（重复挂链会导致循环链表断裂或形成错误环）。
- 入队后维护 `proc->rq` 指向当前队列，并更新 `proc_num`，保证队列元数据一致。

③ FIFO_dequeue：出队（删除指定进程）

```
static void FIFO_dequeue(struct run_queue *rq, struct proc_struct *proc) {
    assert(!list_empty(&(proc->run_link)) && proc->rq == rq);
    list_del_init(&(proc->run_link));
    rq->proc_num--;
}
```

分析：

- 出队前断言两个条件：该进程确实在某个队列中（`run_link` 非空）且属于当前运行队列（`proc->rq == rq`），避免“从错误队列移除”或“重复出队”。
- 使用 `list_del_init` 而不是 `list_del`：删除后会把节点重新初始化为自环，这样能继续用 `list_empty(&proc->run_link)` 判断“是否在队列中”，并确保该进程后续可以安全再次入队（不会残留悬空指针）。
- 最后 `proc_num--` 维护计数正确。

④ FIFO_pick_next：选择下一个进程（取队首）

```
static struct proc_struct *FIFO_pick_next(struct run_queue *rq) {
    list_entry_t *le = list_next(&(rq->run_list));
    if (le != &(rq->run_list)) {
        return le2proc(le, run_link);
    }
    return NULL;
}
```

分析：

- FIFO 的“下一进程”就是队首元素，因此直接取 `list_next(head)`；如果返回的还是 `head`，说明队列为空，返回 `NULL`。
- `le2proc(le, run_link)` 把链表节点转换为进程指针（从而交给框架完成真正的 context switch）。
- 该实现的边界情况非常清晰：空队列返回 `NULL`，单元素/多元素都能正确取队首。

⑤ FIFO_proc_tick：时钟中断（非抢占式）

```
static void FIFO_proc_tick(struct run_queue *rq, struct proc_struct *proc) {
}
```

分析：

FIFO 是典型的非抢占式策略：只要进程开始运行，就会一直运行，直到它主动让出 CPU（如阻塞/睡眠）或退出。因此在 tick 中不需要像 RR/Stride 那样递减时间片、更不需要设置 `need_resched` 触发抢占调度；换句话说，时钟中断不会改变当前运行进程的“继续执行权”。这种设计实现简单、开销低，但缺点是交互性和响应性较差（一个 CPU-bound 进程可能长时间占用 CPU）。

⑥ 调度类注册：fifo_sched_class

```
struct sched_class fifo_sched_class = {
    .name = "FIFO_scheduler",
    .init = FIFO_init,
    .enqueue = FIFO_enqueue,
    .dequeue = FIFO_dequeue,
    .pick_next = FIFO_pick_next,
    .proc_tick = FIFO_proc_tick,
};
```

分析：

该结构体将 FIFO 的五个核心接口挂接到 uCore 的调度框架。完成注册后，只需在 `sched_init()` 中切换 `sched_class` 指针即可启用 FIFO，框架层（`wakeup_proc/schedule` 等）无需额外修改，体现了“框架与策略解耦”的设计。

(2) FIFO 测试代码与测试结果分析（正确性验证）

接下来我将给出 FIFO 的测试程序，说明测试思路；结合 QEMU 输出截图分析调度现象，证明 FIFO 实现正确。

① 测试程序 (fork 多子进程 + 计算负载 + wait 回收)

```
#include <stdio.h>
#include <ulib.h>

#define MAX_CHILDREN 5

int main(void) {
    int i, pid;
    cprintf("FIFO Test Started\n");

    for (i = 0; i < MAX_CHILDREN; i++) {
        if ((pid = fork()) == 0) {
            cprintf("Child %d running\n", i);
            int j;
            for (j = 0; j < 10000000; j++);
            cprintf("Child %d finished\n", i);
            exit(0);
        }
        cprintf("Parent forked child %d\n", i);
    }

    cprintf("Parent waiting for children...\n");
    for (i = 0; i < MAX_CHILDREN; i++) {
        wait();
    }
    cprintf("FIFO Test Finished\n");
    return 0;
}
```

测试思路说明：

- 父进程按 `i=0..4` 顺序创建 5 个子进程；每个子进程打印 `running` 后进行较长空循环模拟 CPU 计算，再打印 `finished` 并退出。
- FIFO 的目标现象应当是：父进程 fork 完后进入 `wait()`，随后子进程按创建顺序 **0→1→2→3→4** 依次获得 CPU；并且由于 FIFO 非抢占，每个子进程的 `running` 与 `finished` 应当连续成对出现，中间不应被其他子进程输出穿插。

② QEMU 输出截图

```
kernel_execve: pid = 2, name = "test_fifo".
FIFO Test Started
Parent forked child 0
Parent forked child 1
Parent forked child 2
Parent forked child 3
Parent forked child 4
Parent waiting for children...
Child 0 running
Child 0 finished
Child 1 running
Child 1 finished
Child 2 running
Child 2 finished
Child 3 running
Child 3 finished
Child 4 running
Child 4 finished
FIFO Test Finished
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:538:
  initproc exit.
```

③ 现象分析

- 截图中先出现：
 - FIFO Test Started
 - Parent forked child 0..4
 - Parent waiting for children...

说明父进程先顺序创建所有子进程，随后进入等待（阻塞），触发调度器选择就绪队列中的子进程运行。
- 随后子进程输出严格按顺序出现：
 - Child 0 running → Child 0 finished
 - Child 1 running → Child 1 finished
 - ...

- Child 4 running → Child 4 finished

这直接验证 FIFO 的“先来先服务”：就绪队列按入队顺序（fork 顺序）依次出队运行。

- 同时，每个子进程的 `running` 与 `finished` 都是连续出现、没有与其他子进程交错穿插，符合 FIFO 的非抢占特性：在 `FIFO_proc_tick` 中不设置 `need_resched`，因此进程不会因为时钟 tick 被强制切走，而是一直运行到完成循环并 `exit(0)`。
- 最后父进程输出 `FIFO Test Finished`，说明所有子进程均已退出并被 `wait()` 正常回收，测试流程闭环结束。

(3) SJF 调度算法实现分析

接下来我将给出 SJF 调度器的关键实现代码，并按函数说明设计思路、关键点与边界情况处理。

① SJF_init：初始化运行队列

```
static void SJF_init(struct run_queue *rq) {
    list_init(&(rq->run_list));
    rq->proc_num = 0;
}
```

分析：

SJF 同样使用 `run_list` 作为就绪队列的基础数据结构，因此初始化逻辑与 FIFO 一致：将 `run_list` 初始化为空循环链表，并将 `proc_num` 清零。这样后续可以通过 `list_next(head)==head` 的方式判定空队列，避免 `pick_next` 在空队列上解引用导致错误。

② SJF_enqueue：入队（简单插入队尾，不在入队阶段排序）

```
static void SJF_enqueue(struct run_queue *rq, struct proc_struct *proc) {
    assert(list_empty(&(proc->run_link)));
    list_add_before(&(rq->run_list), &(proc->run_link));
    proc->rq = rq;
    rq->proc_num++;
}
```

分析：

- 本实现选择“入队不排序，出队选择时再遍历找最短”的策略：所有新就绪进程统一插入队尾（`list_add_before(&rq->run_list, ...)`），保持 $O(1)$ 的入队开销。
- `assert(list_empty(&proc->run_link))` 的作用与 FIFO 相同：防止重复入队破坏链表。
- 更新 `proc->rq` 与 `proc_num`，保证进程与队列的关联关系正确。

说明：理论上也可以在 enqueue 阶段按 burst time 插入有序链表（入队 $O(n)$ ，`pick_next` $O(1)$ ）。本实现把代价放在 `pick_next` ($O(n)$)，逻辑更直观、实现更简单。

③ SJF_dequeue：出队（删除指定进程）


```
static void SJF_dequeue(struct run_queue *rq, struct proc_struct *proc) {
    assert(!list_empty(&(proc->run_link)) && proc->rq == rq);
    list_del_init(&(proc->run_link));
    rq->proc_num--;
}
```

分析：

与 FIFO 相同：出队前断言进程确实在当前队列中，使用 `list_del_init` 删除并重置节点，避免悬空/重复删除问题；同时维护 `proc_num` 计数正确。这样进程后续再次变为 RUNNABLE 时能够安全 enqueue。

④ SJF_pick_next：选择下一个进程（核心：遍历链表找“最短作业”）

```
static struct proc_struct *SJF_pick_next(struct run_queue *rq) {
    list_entry_t *le = list_next(&(rq->run_list));
    if (le == &(rq->run_list)) {
        return NULL;
    }

    struct proc_struct *min_proc = le2proc(le, run_link);
    struct proc_struct *p;

    while ((le = list_next(le)) != &(rq->run_list)) {
        p = le2proc(le, run_link);
        if (p->lab6_priority < min_proc->lab6_priority) {
            min_proc = p;
        }
    }

    return min_proc;
}
```

分析：

- SJF 的核心是“选择预计运行时间（CPU burst time）最短的进程”。你的实现用 `lab6_priority` 充当 burst time 的度量，并在 `pick_next` 中遍历整个 `run_list` 找到最小值进程 `min_proc`。
- 边界情况：如果队列为空（`list_next(head)==head`），直接返回 `NULL`，避免错误访问。
- 复杂度：每次调度选择都需要遍历一次链表，因此 `pick_next` 是 $O(n)$ 。但由于实现简单直观，适合实验场景；若要优化可改为“入队阶段保持有序”或使用小根堆/平衡树维护最小值。

说明：我实现选择的是非抢占式 SJF（也称 SPN），即只在发生调度事件时选“当前最短作业”，并不会因为更短作业到来就抢占正在运行的进程（抢占版称 SRTF）。

⑤ SJF_proc_tick：时钟中断（非抢占式）

```
static void SJF_proc_tick(struct run_queue *rq, struct proc_struct *proc) {
}
```

分析：

SJF 在本实验实现为**非抢占式**：一旦某个进程被选中运行，就会持续运行到它主动阻塞/退出才切换。因此 tick 不需要维护时间片，也不需要设置 `need_resched` 触发抢占。这样能体现 SJF “一次选定，跑到结束/阻塞” 的策略特征，但缺点是交互性可能较差，并且存在“长作业饥饿”风险（短作业不断到来会导致长作业长期得不到调度）。

⑥ 调度类注册：sjf_sched_class

```
struct sched_class sjf_sched_class = {
    .name = "SJF_scheduler",
    .init = SJF_init,
    .enqueue = SJF_enqueue,
    .dequeue = SJF_dequeue,
    .pick_next = SJF_pick_next,
    .proc_tick = SJF_proc_tick,
};
```

分析：

该结构体将 SJF 的核心接口挂接到 uCore 调度框架。切换调度算法时只需要在 `sched_init()` 指向 `sjf_sched_class`，其余框架代码（`wakeup_proc/schedule` 等）无需修改，体现了调度框架与具体策略的解耦设计。

（4）SJF 测试代码与测试结果分析（正确性验证）

接下来我将给出 SJF 的测试程序，说明测试思路；结合 QEMU 输出截图分析调度现象，证明 SJF 实现正确。

① 测试程序（fork 多子进程 + 设置 burst/priority + yield 让所有子进程入队）

```
#include <stdio.h>
#include <ulib.h>

#define MAX_CHILDREN 5

int main(void) {
    int i, pid;
    cprintf("SJF Test Started\n");

    for (i = 0; i < MAX_CHILDREN; i++) {
        if ((pid = fork()) == 0) {
            uint32_t priority = MAX_CHILDREN - i;
            lab6_setpriority(priority);
            cprintf("Child %d created with priority %d\n", i, (int)priority);

            yield();

            cprintf("Child %d executing\n", i);

            int j;
```

```

        for (j = 0; j < 10000000; j++);

        cprintf("Child %d finished\n", i);
        exit(0);
    }
    cprintf("Parent forked child %d\n", i);
}

cpprintf("Parent waiting for children...\n");
for (i = 0; i < MAX_CHILDREN; i++) {
    wait();
}
cpprintf("SJF Test Finished\n");
return 0;
}

```

测试思路说明：

- 本实验将 `lab6_priority` 视为“作业长度/CPU burst time”的度量，数值越小表示作业越短；SJF 的预期行为是“越短越先运行”。
- 父进程创建 5 个子进程 `child 0..4`，每个子进程分别设置优先级 (burst) 为 `5,4,3,2,1`。
- 每个子进程在设置完 `priority` 后调用一次 `yield()`：其目的不是让出 CPU “随机跑”，而是确保所有子进程都完成 `priority` 设置并进入就绪队列，这样调度器在真正做选择时，队列里已经有完整的候选集合，能体现 SJF 的“全局挑最短”。
- 因此若 SJF 实现正确，则应该观察到执行顺序为：priority 最小的先执行，即 `priority=1` 的 **Child 4** 最先执行，然后 **Child 3**、**Child 2**、**Child 1**、**Child 0** 依次执行。

② QEMU 输出截图

```
kernel_execve: pid = 2, name = "test_sjf".
SJF Test Started
Parent forked child 0
Parent forked child 1
Parent forked child 2
Parent forked child 3
Parent forked child 4
Parent waiting for children...
set priority to 5
Child 0 created with priority 5
set priority to 4
Child 1 created with priority 4
set priority to 3
Child 2 created with priority 3
set priority to 2
Child 3 created with priority 2
set priority to 1
Child 4 created with priority 1
Child 4 executing
Child 4 finished
Child 3 executing
Child 3 finished
Child 2 executing
Child 2 finished
Child 1 executing
Child 1 finished
Child 0 executing
Child 0 finished
SJF Test Finished
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:538:
  initproc exit.
```

③ 现象分析

- 截图中父进程先输出 `Parent forked child 0..4`，随后进入 `Parent waiting for children...`。父进程进入等待会阻塞，从而让调度器开始从就绪队列中选择子进程运行，这保证了“子进程是主要被调度对象”。
- 接下来可以看到每个子进程依次设置 priority（从 5 到 1），对应输出：

- `Child 0 created with priority 5`
- `Child 1 created with priority 4`
- `Child 2 created with priority 3`
- `Child 3 created with priority 2`
- `Child 4 created with priority 1`

这说明测试用例确实把“不同作业长度”注入到了调度系统中。

- 更关键的是随后执行顺序（`executing/finished`）严格为：

- `Child 4 executing` → `Child 4 finished`
- `Child 3 executing` → `Child 3 finished`
- `Child 2 executing` → `Child 2 finished`
- `Child 1 executing` → `Child 1 finished`
- `Child 0 executing` → `Child 0 finished`

这与 SJF “最短作业优先” 完全一致：priority 越小（burst 越短）的进程越先被 `SJF_pick_next` 选中。

- 同时，每个 child 的 `executing` 与 `finished` 基本成对出现，符合本实验实现的非抢占式 SJF：一旦进程被选中，tick 不触发抢占，它会一直运行到完成循环并退出。

结论：QEMU 输出清晰体现了“priority=1 的 Child 4 最先执行，随后依次 priority=2,3,4,5”，与非抢占式 SJF 的设计完全一致，证明 SJF 调度器实现正确。末尾 `initproc exit` 的 panic 为 uCore 实验常见收尾现象，不影响调度正确性判断。